

Reviewer Pack — Minimal Index of Quasigroups and Communication Complexity Ra...

Entry 85495192daa0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:22:59 UTC

Minimal Index of Quasigroups and Communication Complexity Rank

Entry ID: 85495192daa0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:22:59 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (Quasigroups) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

The minimal index of a quasigroup Q is linearly correlated with the communication complexity rank of any matrix that can be represented by Q , such that $\min_index(Q) = \Omega(\text{rank}(\text{matrix Represented by } Q))$ and $\max_index(Q) = O(\text{rank}(\text{matrix Represented by } Q))$.

Rationale (proposer's reasoning):

Quasigroups are fundamental algebraic structures with a rich combinatorial nature. Their minimal index captures their essential complexity, and it may provide insights into the communication complexity of matrices that can be represented by these structures. The potential for a correlation suggests a bridge between abstract algebra and computational complexity.

Taxonomy category: COMBINATORIAL_ALGEBRA_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 06239927e2c3bf46

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For quasigroups Q with size $n \leq 40$, if the correlation coefficient between $\min_index(Q)$ and $\text{rank}(\text{matrix Represented by } Q)$ is greater than or equal to 0.8 AND the mean of metric_mean across all seeds is less than or equal to 3, then support is provided for the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_quasigroup(n):
        q = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                q[i][j] = (i + j) % n
        return q

    def min_index(q):
        n = len(q)
        indices = [0] * n
        for i in range(n):
            for j in range(n):
                if q[i][j] == 0:
                    indices[i] += 1
        return max(indices)

    def communication_complexity_rank(matrix):
        n = len(matrix)
        eigenvalues = []
        for k in range(2, n + 1):
            A_k = [[matrix[i][j] ** k for j in range(n)] for i in range(n)]
            A_k_sum = sum(sum(row) for row in A_k)
            if A_k_sum == 0:
                continue
            eigenvalues.append(A_k_sum / (n * n))
```

```

    return len(eigenvalues)

def matrix_from_quasigroup(q):
    n = len(q)
    matrix = [[int(q[i][j] == k) for j in range(n)] for k in range(n)]
    return matrix

n_values = [5, 10, 15, 20, 30, 40]
metric_sum = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Sample 5 instances per size
        q = generate_quasigroup(n)
        matrix = matrix_from_quasigroup(q)
        min_index_val = min_index(q)
        rank_val = communication_complexity_rank(matrix)

        if rank_val == 0:
            continue

        metric_sum += min_index_val / rank_val
        instances_tested += 1
        n_max = max(n_max, n)

mean_metric_value = metric_sum / instances_tested

return {
    "metric_name": "min_index_over_rank",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")

```

```

else:
    for r in results:
        if not r["conjecture_holds"]:
            counterexample = f"n={r['instances_tested']}, min_index_over_rank={r['metric_value']}"
            break
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[re

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12c2eec3.py", line 90, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12c2eec3.py", line 62, in run_trial
    matrix = matrix_from_quasigroup(q)
              ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12c2eec3.py", line 49, in matrix_from_q
    matrix = [[int(q[i][j] == k) for j in range(n)] for k in range(n)]
              ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data or results that could be used to support or falsify the conjecture. | next: Review and correct the error in the test code that caused the crash, then rerun the test with a valid quasigroup input to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18254
2	propose	ollama_remote	glm4:latest	0	0	12593
3	preregistration	ollama_remote	glm4:latest	0	0	9282
4	novelty	ollama_remote	glm4:latest	0	0	8616
5	novelty	ollama_remote	glm4:latest	0	0	8424
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16280

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10935
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13054
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10930
10	judge	ollama_remote	glm4:latest	0	0	12679

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 121047 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/85495192daa0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/85495192daa0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/85495192daa0.tar.gz (if generated)